

Movie Reservation

과목 : 데이터베이스 05분반
교수 : 이성훈
학과 : 응용통계학과
소프트웨어학부
산업보안학과
20204885
학번 : 20240776
20215653
강태영(조장)
이름 : 왕규원
이규현

1. 개요(Introduction)

1.1. 프로젝트 주제

영화 예매 시스템(Movie Reservation System)

1.2. 프로젝트 개발 목적

영화 예매 서비스의 실제 운영 구조(로그인 → 영화 선택 → 상영 선택 → 좌석 선택 → 결제)를 분석하고, 이를 기반으로 한 데이터베이스 설계 및 프로그래밍 구현을 주목적으로 하였다.

실제 영화관 서비스(롯데시네마, CGV, 메가박스 등)의 기능을 참고하여 현실적인 도메인 모델을 구성하였으며, 사용자 경험(UX)을 고려한 프로세스를 설계하였다.

본 프로젝트는 데이터베이스 05분반(CAU CS:59190)의 기말 프로젝트로 진행되었으며, 대규모 스키마(30개 이상 테이블) 설계, 함수/프로시저/트리거의 활용, Java 기반의 CRUD 구현 능력 강화를 목표로 하였다.

1.3. 프로젝트 개요

- Java 기반 콘솔 애플리케이션(IntelliJ + MySQL)
- JDBC를 통한 Java-DB 연동
- DAO-Service-Main으로 구성된 3-Tier 아키텍처

1.4. 실제 구현 범위

제한된 개발 기간 내에 완성도를 높이기 위해, 전체 기능 중 실제 서비스의 핵심이 되는 기능을 선별하여 구현 범위를 설정하였다.

- 회원 관리 : 회원 가입 및 로그인(이메일 기반)
- 조회 기능 : 영화/상영 정보 조회, 상영관 및 좌석 배치도 조회
- 예매 프로세스 : 좌석 선택, 좌석 홀드를 통한 동시 접근 제어, 예매 처리
- 결제 및 환불 : 결제 상태 관리, 예매 취소 및 환불
- 시스템 관리 : 주요 데이터 변경에 대한 로그 자동 기록

1.5. 데이터베이스 구성

총 42개의 테이블로 구성되었으며, 영화/상영/좌석/가격 정책/회원/비회원/결제/스낵 서비스 등 실제 데모 서비스가 가능할 정도의 세분화된 스키마를 설계하였다.

복잡한 비즈니스 로직 처리를 위해 함수, 프로시저, 트리거를 활용하였다.

1.6. 개발 환경

- IDEA : IntelliJ
- JDK : 23.0.2
- MySQL : 8.0.43

2. 요구사항(Requirements)

2.1. 기능 요구사항

2.1.1. 회원 관리 기능

- 이메일 기반 회원 가입 및 로그인
- 회원 정보와 예매 내역을 조회(회원만 구현)
- 등급은 누적 사용 금액에 의해 결정되며, 프로시저(sp_recalc_member_tier)를 통해 처리

2.1.2. 영화 및 상영 정보 조회 기능

- (상영 중) 영화 목록과 각 영화의 상세 정보(제목, 상영시간, 배급사, 연령 등급, 상영방식(2D/3D/IMAX) 등)가 포함된다.
- 영화 선택 이후 사용자에게 상영 시간표를 조회 기능 및 상영관, 영화 버전을 함께 제공.

2.1.3. 좌석 조회 및 선택 기능

- 상영관의 좌석 배치, 좌석 유형, 가격 차이 등을 확인 가능
- 예약된 좌석은 선택할 수 없음
- 좌석 상태를 실시간으로 확인하여 선택 가능 여부 확인 가능

2.1.4. 예매 기능

- 좌석 선택 후 예매 가능
- 최종 금액은 가격 정책(price_rule)과 좌석 유형(seat_type)을 기반으로 계산
- 예매 시 booking, booking_seat 테이블에 데이터가 저장(초기 상태 PENDING)
- 예매 전 홀드 된 좌석은, 새로운 예매 시 정리됨

2.1.5. 결제 기능

- 결제 수단(payment_method)를 선택하여 결제
- 결제 완료 시 payment 테이블에 결제 정보 저장
- 결제 성공시 트리거(trg_payment_after_insert)가 자동 실행되어 예매 상태를 갱신하고 로그(audit_log)를 기록

2.1.6. 예매 취소 및 환불 기능

- 상영 20분전까지 예매 취소 가능
- 환불 여부와 금액은 함수(fn_calc_refund_amount)를 통해 계산
- 취소 시 refund 정보가 기록되며, booking 상태는 취소로 변경됨

2.1.7. 감사 로그 및 기록 기능

- 시스템은 주요 이벤트에 대해서 로그를 기록함
- 로그에는 주체, 동작, 테이블, 상세 정보 등이 포함됨
- 트리거를 활용해서 시스템 수준에서 기록이 자동 생성

2.2. 비기능 요구사항

2.2.1. 데이터베이스

- 42개의 테이블
- 정규화 원칙, 관계 무결성 보장을 위해 PK, FK, UNIQUE 적용
- 주요 비즈니스 로직의 일부는 함수, 프로시저, 트리거를 활용하여 DB에서 처리

2.2.2. 구조

- Java 기반 콘솔
- DAO -> Service -> Main으로 3-Tier
- JDBC를 사용하여 DB 연결 및 예외 처리와 입력값 검증이 수행됨

2.2.3. 동시성

- 홀드 상태의 좌석은 선택할 수 없음(중복 방지)
- 만료된 홀드 상태는 sp_cleanup_expired_hold_seat을 통해 주기적으로 제거
- 예매 및 결제 처리 과정에서 데이터 일관성 보장(트랜잭션 활용)

2.2.4. 확장성(추후 확장 가능)

- 쿠폰 및 프로모션 할인 처리
- 비회원 기능
- 리뷰 및 좋아요 기능
- 추천 알고리즘
- 스낵 주문 및 매장 관리
- 알림
- QR코드 기반 발권

2.3. 제약 조건

2.3.1. 개발 환경

- IDEA : IntelliJ + DB : MySQL
- 단일 로컬 서버에서만 실행 가능

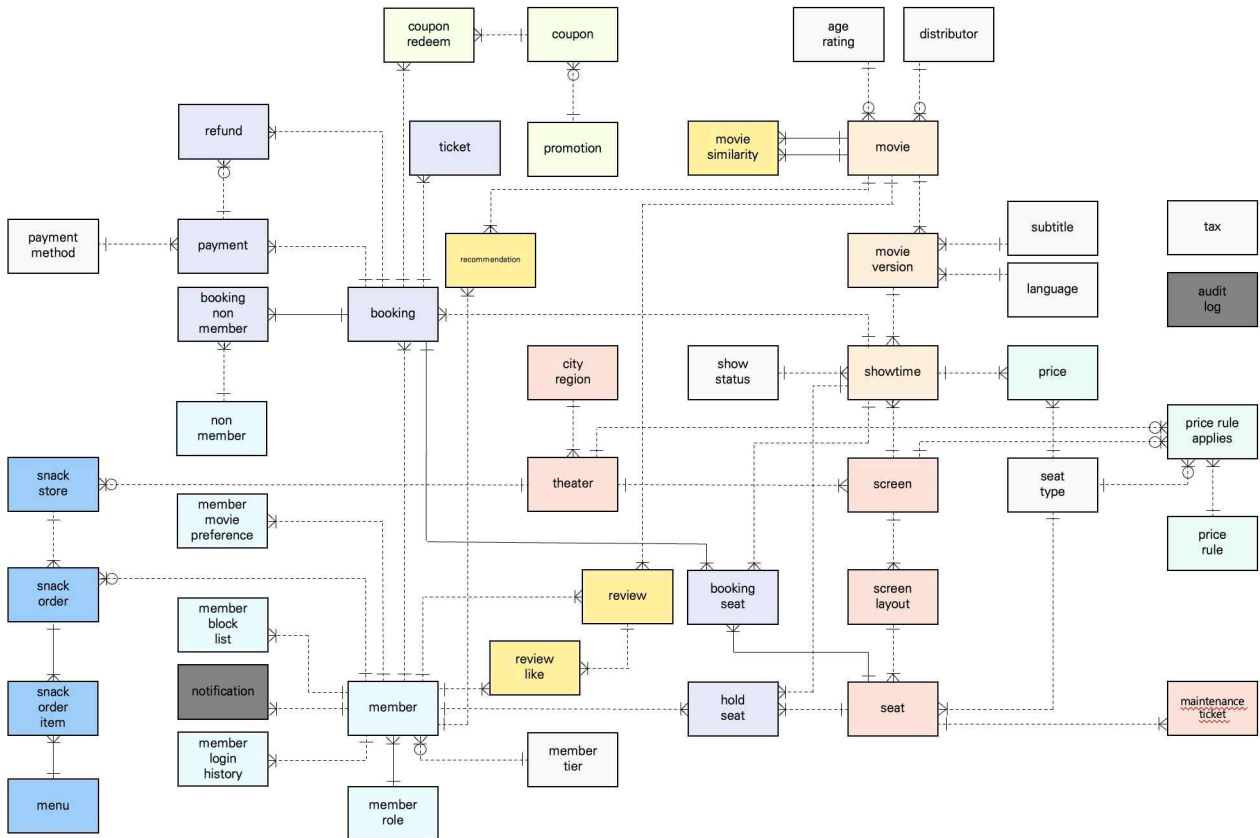
2.3.2. 구현 범위

- 콘솔 환경으로 GUI 환경 제공 X
- 웹/모바일 환경은 고려하지 않음
- 프로젝트 수행 기간의 제약으로 인해, 실무 서비스의 완전한 구현보다 핵심 흐름 및 DB 연동을 위주로 작업
- 일부 엔티티는 스키마에 포함되지만, 프로그램에서는 구현 생략

3. 전체 데이터베이스 설계(DataBase Design)

3.1. IE Notation

42개에 달하는 대규모 테이블 구조의 복잡성을 최소화하기 위해, 관계의 참조가 집중되는 중심 테이블을 다이어그램의 안쪽에 배치하였다. 이를 통해 참조 흐름이 외곽에서 중심으로 향하도록 하고, 관계선의 꼬임이나 교차를 물리적으로 최소화했다.



3.2. 그룹별 설계

테이블을 기능적으로 묶고, 비즈니스 로직을 직관적으로 파악하기 위해, 아래와 같이 **색상별로 그룹화**하였다. 각 그룹은 설계 근거, ER 도출 과정, 유연성, 확장성을 함께 제시한다.

3.2.1. 영화

주요 테이블(살구색 그룹): movie, movie_version

참조 테이블(회색 그룹): distributor, age_rating, language, subtitle_language

- 이 시스템의 모든 데이터 흐름의 출발점이 되는 **핵심 도메인**이다.
- 영화 정보(title, running_time, distributor_id 등)는 독립적인 속성 집합이므로 단일 엔티티로 도출한다.
- movie_version은 IMAX/2D/4DX 등 여러 버전을 가진 특성을 반영해 1:N 관계로 분리하였다.
- 이러한 구조는 새로운 포맷, 언어, 자막이 등장해도 movie 테이블 변경 없이 확장 가능하다.

3.2.2. 상영인프라

주요 테이블(살구색 그룹): movie, movie_version

참조 테이블(회색 그룹): distributor, age_rating, language, subtitle_language

- 영화가 실제로 상영되는 환경을 정의하며, **예매 흐름의 기반**이 되는 중요한 구조이다.
- theater, screen, screen_layout, seat은 물리적 구조를 단계적으로 분리하여 관리한다.
- 레이아웃 교체가 가능하므로 극장 리모델링 시에도 좌석 구조를 유연하게 변경할 수 있다.
- seat_type을 별도 엔티티로 설계하여 VIP/Royal 등 좌석 정책 변경에 대응하도록 했다.

3.2.3. 가격 및 할인 정책

주요 테이블(민트·연두색 그룹): price, price_rule, price_rule_applies, promotion, coupon

참조 테이블(살구·붉은·하늘색 그룹): movie_version, seat_type, member_tier

- 상영, 좌석 타입, 요일, 시간대에 따라 **가격이 달라지는 복잡한 규칙**을 수용한다.
- price_rule은 다양한 규칙을 조합할 수 있는 구조를 제공해 미래 정책에도 대응 가능하다.
- fn_get_final_price()를 통해 DB 내에서 최종 가격을 결정하여 애플리케이션 로직을 단순화했다.

3.2.4. 회원

주요 테이블(민트·연두색 그룹): price, price_rule, price_rule_applies, promotion, coupon

참조 테이블(살구·붉은·하늘색 그룹): movie_version, seat_type, member_tier

- 영화관 시스템의 주체가 되는 **사용자 정보**를 다루는 영역이다.
- 회원은 예매·결제의 상위 개념이므로 핵심 도메인(영화/상영/좌석/가격) 이후에 설명하는 것이 자연스럽다.
- member_role, member_tier 등은 서비스 운영 정책에 따라 확장성을 가진다.

3.2.5. 예매/결제 흐름

주요 테이블(연보라색 그룹): booking, booking_seat, payment, refund, hold_seat

참조 테이블(붉은·민트·하늘색 그룹): showtime, seat, price_rule, member

- 앞서 정의된 모든 도메인을 종합적으로 사용하는 **시스템의 핵심 흐름**이다.
- booking과 booking_seat를 분리하여 다대다 관계를 해소하고 예매 이력을 정상적으로 관리한다.
- 결제는 트리거를 통해 예약 상태를 자동으로 갱신하여 무결성을 유지한다.
- 환불은 fn_calc_refund_amount()를 통한 정책 기반 처리 구조를 가진다.

3.2.6. 리뷰 및 추천

주요 테이블(노란색 그룹): review, review_like, recommendation

참조 테이블(살구·하늘색·연보라색 그룹): movie, member, booking

- 핵심 예매 흐름 이후 **사용자 경험을 확장**하는 기능이다.
- 영화 선호도 분석, 추천 모델 등 확장 가능성을 고려해 별도 그룹으로 분류했다.

3.2.7. 스낵/매점

주요 테이블(파란색 그룹): snack, snack_order

참조 테이블(하늘색·연보라색 그룹): member, payment

- 영화관 비즈니스 확장 기능으로 예매 시스템과는 독립적이지만, 회원 및 결제 도메인과 연동될 수 있다.

3.2.8. 시스템 기본 테이블

주요 테이블(회색 그룹): status, code_table, distributor, age_rating

참조 테이블(전체 도메인): 각 도메인의 FK로 연결되는 공통 코드/정책 테이블

- 주요 도메인들의 관리 및 정책 반영을 위한 보조적 엔티티들이다.
- 정책 변경 시 중앙 집중형 테이블을 통해 일괄 처리 가능하도록 설계했다.

3.2.9. 로깅/알림

주요 테이블(진한 회색 그룹): audit_log, notification

참조 테이블(하늘색·연보라색 그룹): member, booking, payment 등 주요 행위 주체 및 트랜잭션 엔티티

- 모든 기능의 운영 및 감사 기능을 담당하는 시스템적 기반 구조이다.
- 트리거 기반 자동 로그 기록으로 변경 이력을 보존하고 문제 발생 시 추적 가능성을 높였다.

3.2.10. 정리

본 시스템은 “영화·상영·좌석·예매·결제”의 업무 흐름을 중심으로 ERD가 구성된다.

영화는 여러 버전(movie_version)으로 확장되고, 각 버전은 상영관(screen)과 결합되어 하나의 상영(showtime)을 형성한다.

상영관의 좌석 구조(screen_layout, seat)를 기반으로 사용자는 여러 좌석을 선택해 예매(booking)를 생성하며, 결제(payment)·환불(refund)은 booking 단위로 처리된다.

이 흐름은 실제 영화관 운영을 반영하면서 정규화된 스키마 구조를 갖추도록 설계되었다.

3.3. ERD 필드 기반 상세 테이블 설계 설명

아래 내용은 기존 ERD가 필드 나열 중심으로 표현되어 설명이 부족했던 부분을 보완하기 위해, 각 핵심 테이블이 어떤 목적·유연성·기능적 가능성을 염두에 두고 설계되었는지를 정리한 것이다.

3.3.1. movie (영화 정보) 테이블

핵심 필드: movie_id, title, running_time, distributor_id, age_rating_id 등

- distributor_id는 별도 테이블로 분리하여 추후 새로운 배급사 추가 시 movie 테이블 구조 변경이 필요 없도록 하여 높은 확장성을 확보하였다.
- age_rating_id를 외래키로 두어 등급 정책 변경 시 모든 영화 데이터를 수정할 필요 없어 정책 변경 유연성이 높다.
- 영화 자체 속성은 원자적이므로 정규화 원칙을 자연스럽게 충족하였다.

3.3.2. movie_version (IMAX/2D/4DX/자막 등) 테이블

핵심 필드: version_id, movie_id, format, language_id, subtitle_id 등

- 영화와 분리된 이유는 한 영화가 여러 버전(IMAX, 2D 등)을 갖기 때문에 1:N 구조가 자연스럽게 중복 데이터를 잘 처리하게 하였다.
- language_id, subtitle_id를 분리하여 다국어/자막 정책을 자율적으로 확장 가능하게 하였다.
- format 필드에 IMAX/4DX 등 새로운 포맷 등장 시에도 테이블 구조 변경 없이 데이터만 추가하면 되므로 미래 확장성이 강화되었다.

3.3.3. showtime (상영 정보) 테이블

핵심 필드: show_id, movie_id, version_id, screen_id, start_time 등

- 영화/버전/상영관을 조합하여 “하나의 상영 이벤트”를 정의하여 실제 영화관 운영 모델 그대로 반영하였다.
- 스크린(screen)과 직결되어 좌석 배치도 추적 가능하다.
- start_time 기반으로 예매 가능 여부와 취소 가능 시간(20분 전) 등 규칙 검사 가능하여 비즈니스 로직 친화적인 구조를 형성한다.

3.3.4. seat / screen_layout 테이블

핵심 필드: seat_id, layout_id, row_label, col_number, seat_type_id 등

- seat_type을 분리함으로써 VIP/Royal/Standard 등 좌석 가격 정책을 독립적으로 관리할 수 있다.
- 좌석 자체는 layout에 종속되어 있어 스크린 구조 변경 시 layout 교체만으로 전체 좌석 재구성 가능하여 물리적 인프라 변화에 강하게 구성되었다.
- row + col 조합으로 정렬 출력이 쉬워 콘솔/GUI 표시 모두 최적화되었다.

3.3.5. price / price_rule / price_rule_applies 테이블

핵심 필드: price_id, 좌석/상영/회원 등과 연결되는 FK들, percentage_amount, flat_amount, 적용 조건 속성들(rule_type, target_type 등)

- 다양한 할인 및 가격 정책을 규칙 기반(price_rule)으로 분리하여 가격 정책의 완전한 유연성을 확보하였다.
- seat_type, 요일, 시간대, 영화 버전(IMAX 등)에 따른 변화도 rule로 작성 가능하여 쿠폰·프로모션 확장도 가능하다.
- DB 함수 fn_get_final_price()에서 rule을 조합적으로 적용하여 애플리케이션 로직을 단순화하였다.

3.3.6. booking / booking_seat 테이블

핵심 필드: booking_id, member_id, show_id, status, total_amount, seat_id, price 등

- 다대다 해결 구조로, 한 번의 예매에서 여러 좌석을 처리할 수 있다.
- 좌석 가격(price)을 booking_seat에 저장하는 이유는 예매 시점 가격을 고정 저장하여 추후 가격 정책 변경과 무관하게 과거 예매 이력 보존을 위함이다.
- booking.status(PENDING/CONFIRMED/CANCELLED)는 결제/환불 트리거에 의해 자동 갱신된다.

3.3.7. hold_seat 테이블

핵심 필드: showtime_id, seat_id, held_by, expires_at, created_at 등

- 웹/콘솔 기반 예매 서비스에서 중복 접근을 막기 위한 핵심 테이블이다.
- expires_at이 존재하여 사용자가 선택만 하고 결제하지 않아도 일정 시간이 지나면 자동 반환된다.
- 서비스 확장 시 Redis 기반 좌석 캐싱 시스템으로도 발전 가능하므로 구조적 유연성을 보유한다.

3.3.8. payment / refund 테이블

핵심 필드: payment_id, booking_id, amount, method, paid_at, status / refund_id, payment_id, refund_amount, reason, refunded_at

- payment(insert) 시마다 트리거로 booking 상태 변경과 로그 기록이 자동으로 남아 데이터 무결성과 추적성을 확보하였다.
- refund는 payment 기반의 1:1 구조로 설계되어 중복 환불 방지가 가능하다.

3.4. 정규화(NF) 과정

각 테이블이 정규형(1NF → 2NF → 3NF)을 만족하는지 함수 종속 중심으로 검증한 요약이다.

3.4.1. Booking 엔티티 정규화 요약

- PK : booking_id
- 1NF : 모든 속성은 원자값(paid_at, status, booked_at 등)이다.
- 2NF : 기본키는 단일 속성이므로 부분 함수 종속이 존재하지 않는다.
- 3NF : member_id, showtime_id, status, total_amount 간 이행적 함수 종속이 없도록 금액 계산 및 상태 변경은 price_rule, payment 등 외부 엔티티에서 처리된다.

3.4.2. BookingSeat 엔티티

- PK : (booking_id, seat_id)
- 1NF : 가격(price)은 좌석당 하나의 원자값이다.
- 2NF : 모든 속성이 복합키 전체에 종속되며 booking_id 또는 seat_id 한쪽에만 종속되는 속성이 없다.
- 3NF : price는 예매 시점의 좌석 가격이며 다른 속성에 의해 결정되지 않아 이행 종속이 없다.

3.4.3. Price / PriceRule 엔티티

- PK : price_id, rule_id
- 1NF : 할인 유형, 금액, 좌석 타입 등 모든 속성은 원자적이다.
- 2NF : 단일 기본키 구조로 부분 종속이 없다.
- 3NF : 좌석 타입·요일·상영 버전 등에 따른 가격 변화는 price_rule로 분리되어 이행 종속이 제거된다.

3.4.4. Movie / MovieVersion 엔티티

- PK : movie_id / version_id
- 1NF : title, running_time 등 모든 속성은 원자값이다.
- 2NF : movie_version은 version_id에 완전 종속되며 movie_id에 대한 부분 종속이 없다.
- 3NF : IMAX/2D/4DX 등 버전별 속성이 movie에 이행적으로 종속되지 않도록 별도 엔티티로 분리되어 정규화되었다.

4. 시스템 설계 및 구현(System Design & Implementation)

4.1. 시스템 전체 구조

본 프로젝트는 영화 예매 서비스를 3-Tier 기반으로 설계하였다.

- 사용자 인터페이스(Presentation Layer)

콘솔 기반 환경이며, Main.java가 시스템의 진입점이다. 사용자 입력을 처리하고 메뉴로 흐름을 제어한다. 비즈니스 로직 호출에 필요한 데이터를 수집하여 Service 계층에 전달한다.

구성 : Main

- 서비스(Service Layer)

시스템의 모든 사례(예매 생성, 결제 처리 등)를 담당하는 핵심 계층이다. 복수의 DAO를 조합하여 하나의 업무를 처리한다. 좌석 사용 상태 검증, 예매 생성 등과 같은 핵심 비즈니스 로직을 이곳에서 수행한다. 데이터베이스 오류나 비즈니스 로직 위반을 처리한다.

구성 : service/BookingService, service/MemberService, etc.

- DAO(Data Access Object)

MySQL에 직접적인 데이터 접근을 담당한다. 각 DAO는 단일 테이블 혹은 단일 쿼리 단위로 분리되어 있다. CRUD, JOIN 중심의 SQL을 수행한다. JDBC 기반으로, PreparedStatement, ResultSet을 활용하여 데이터 매핑을 수행한다.

구성 : dao/BookingDao, dao/SeatDao, etc.

- DataBase

총 42개의 테이블로 구성된 스키마는 실제 영화관 예매 서비스 도메인을 반영하여 설계되었으며, 모든 기능을 데이터 모델로 표현한다. 주요 비즈니스 로직의 일부는 DB에서 함수, 프로시저, 트리거 형태로 수행된다.

함수 : fn_get_final_price(가격 계산), fn_calc_refund_amount(환불금)

프로시저 : sp_recalc_member_tier(회원 등급 갱신), sp_cleanup_expired_hold_seat(홀드 좌석 정리)

트리거 : trg_payment_after_insert(결제 상태 변경), trg_booking_update_audit(감사로그 기록)

- 계층 간 연동 방식

인터페이스 → 서비스 → DAO → DB 순서로 흐름을 유지한다. 각 계층은 인접한 계층 간만 접근할 수 있다.

4.2. 프로그램 설계

```
org.example/
├─ Main.java # 진입점 + 콘솔 UI
├─ util/
│   └─ DBUtil.java # DB 연결 (환경변수 기반)
├─ model/# 엔티티 클래스 (13개)
│   ├── Member.java
│   ├── Movie.java
│   ├── Showtime.java
│   ├── Seat.java / SeatStatus.java (enum)
│   ├── Booking.java / BookingSeat.java
│   ├── HoldSeat.java
│   ├── Payment.java
│   ├── Theater.java / Screen.java
│   ├── Price.java / MovieVersion.java
│   └─ ...
├─ dao/# Data Access Object (12개)
│   ├── MemberDao.java# CRUD: member
│   ├── MovieDao.java # SELECT: movie + distributor JOIN
│   ├── ShowtimeDao.java# SELECT: showtime + movie_version JOIN
│   ├── SeatDao.java# SELECT: seat + booking_seat + hold_seat JOIN
│   ├── BookingDao.java # INSERT: booking
│   ├── BookingSeatDao.java # INSERT/SELECT: booking_seat
│   ├── HoldSeatDao.java# CRUD: hold_seat
│   ├── PaymentDao.java # INSERT: payment
│   ├── PriceDao.java # SELECT: price + fn_get_final_price 호출
│   ├── TheaterDao.java # SELECT: theater
│   ├── ScreenDao.java# SELECT: screen
│   └─ MovieVersionDao.java
├─ service/# 비즈니스 로직 (10개)
├─ MemberService.java# 로그인/회원가입 로직
├─ MovieService.java # 영화 조회
├─ ShowtimeService.java# 상영 시간 조회
├─ SeatService.java# 좌석 조회 + 가용성 체크
├─ BookingService.java # 예매 생성 (핵심 비즈니스 로직)
├─ TheaterService.java
├─ ScreenService.java
└─ ...
```

4.3. 핵심 비즈니스 흐름

4.3.1. 영화/상영/좌석 조회

- **영화 목록 조회** : 사용자 요청 시 MovieService가 MovieDao를 통해 영화 목록을 조회, 필요시 배급사와 연령 등급을 JOIN 하여 상세 정보 제공
- **상영 시간표 조회** : 영화 ID기반으로 ShowtimeService가 showtime을 조회(-> :M movie_id와 screen_id, theater_id 조건 조합에 따라 상영 가능한 시간대를 필터링 하여 제공하며, 사용자의 예매 흐름을 영화 > 지점 > 시간대 순서로 자연스럽게 조회 흐름을 구현), 상영관/영화 버전/언어, 자막 정보를 함께 제공
- **실시간 좌석 상태 조회** : :M SeatService가 해당 상영관의 좌석 배치(layout)를 조회하고 상태(booking_seat, hold_seat)를 활용하여 실시간으로 상태를 확인한다. :+ 좌석 조회 기능은 seat, hold_seat, booking_seat 테이블을 조합하여 현재 시점의 좌석 상태를 도출한다. 동일한 좌석에 대해

booking.status가 CONFIRMED이면 BOOKED, hold_seat.expires_at > NOW()인 레코드가 존재하면 HOLD, 그 외에는 AVAILABLE로 판단하여 결과를 보여주게 된다. 이는 단순 조회가 아닌 상영 상태 기반 계산형 서비스로 동시성 제어 기능을 바탕으로 한 조회를 가능하게 한다. 이때 Service 계층에서는 최종적으로 좌석 상태를 통합 계산하고, DAO 계층의 경우 단순 데이터 제공만 수행하여 계층 간 역할이 명확히 분리되게 된다.

4.3.2. 예매

1. 검증 : 사용자의 선택 정보(영화, 상영 시간, 좌석 정보)의 유효성을 검증하고, 로그인 상태를 식별한다.
2. 홀드 정리 : HoldSeatDao.deleteExpire()를 호출, sp_cleanup_expired_hold_seat 프로시저를 호출하여 오래된 좌석 홀드가 남아 중복 예매를 방지
3. 좌석 중복 및 가격 검사 : 예약 여부 확인(BookingSeatDao.existsActiveBookingForSeat()) → 홀드 상태 확인(HoldSeatDao.findActiveHold()) → 최종 가격 조회(PriceDao.findFinalPriceBySeat() 호출, fn_get_final_price)로 (상영 가격, 할인 규칙, 좌석확인, 세금까지 한 번에 처리 → :M 상영정보, 좌석 유형, 적용된 가격 정책(할인 or 할증) 등을 반영한 최종금액을 한 번에 처리하여 사용자가 추가 계산할 필요 없이 실시간으로 정확한 결제 금액을 보임.)
4. 데이터 생성
 - 가. Booking : BookingDao.create()에서 booking레코드 생성(PENDING으로 저장)
 - 나. BookingSeat : BookingSeatDao.insertSeat()호출 좌석별 가격이 저장됨
5. 홀드 해제 : 예매 성공 후 HoldSeatDao.deleteByMemberAndSeat() 홀드 삭제

4.3.3. 결제 처리

- 상태 검증 : status가 PENDING이지 않으면 결제 불가
- 결제 정보 저장 : PaymentDao.insert() 호출하여 결제 수단(payment_method), 금액(amount), 시간(approved_at) payment 테이블에 기록
- 트리거 기반 상태 전환 : payment 테이블에 삽입되는 즉시 trg_payment_after_insert 트리거 자동 실행하여 audit_log에 결제 이벤트 기록
- 결과 반환 : 성공 여부 및 결제 번호

4.3.4. 취소 및 환불

- 환불 : 상영 시작 20분 전인지 체크하여 fn_calc_refund_amount(booking_id, cancel_time)으로 환불 가능 금액 계산
- 환불 정보 기록 : RefundDao.insert()로 환불 사유(booking_id, payment_id, reason, refund_amount)와 금액을 기록
- 예매 상태 업데이트 : BookingDao.updateStatus(CANCELLED) 진행하고, trg_booking_update_audit 트리거가 시작되어 로그(audit_log) 기록

4.3.5. 좌석 홀드

중복 방지를 위해 hold_seat 테이블을 활용한다.

1. 사용자가 좌석을 선택하면 해당 좌석을 hold로 저장(유효 시간이 지나면 자동 만료, 예매 시도 시 만료 자동 정리, 다른 사용자가 홀드한 좌석은 홀드 불가)
2. 예매 성공 후 홀드 삭제

4.4. Internal Implementation

4.4.1. dao

4.4.1.1. **BookingDao.java** : 예약(booking) 테이블에 접근하는 DAO 클래스

- **create(Booking booking)** : booking 테이블에 새 예약을 INSERT하고 자동 생성된 booking_id를 반환한다

4.4.1.2. **BookingSeatDao.java** : 예약 좌석(booking_seat) 매핑 정보를 다루는 DAO 클래스

- **insert(BookingSeat bookingSeat)** : booking_seat 테이블에 예약 좌석 정보를 INSERT 한다
- **existsActiveBookingForSeat(Long showId, Long seatId)** : 해당 상영의 특정 좌석에 활성 예약 (PENDING/CONFIRMED)이 있는지 조회
- **insertSeat(Long bookingId, Long showId, Long seatId, BigDecimal price)** : 파라미터를 직접 받아 booking_seat 테이블에 INSERT한다

4.4.1.3. **HoldSeatDao.java** : 좌석 임시 점유(hold_seat) 데이터를 관리하는 DAO 클래스

- **create(HoldSeat holdSeat)** : hold_seat 테이블에 새로운 홀드를 INSERT하고 hold_id를 반환한다
- **findActiveHold(Long showId, Long seatId)** : 아직 만료되지 않은(expires_at > NOW) 특정 좌석의 홀드를 조회
- **deleteById(Long holdId)** : hold_id로 홀드 레코드를 삭제한다
- **deleteExpired()** : 만료된 모든 홀드 레코드를 일괄 삭제한다
- **deleteByMemberAndSeat(Long memberId, Long showId, Long seatId)** : 특정 회원이 잡은 특정 좌석 홀드를 삭제한다
- **map(ResultSet rs)** : ResultSet을 HoldSeat 객체로 변환한다 (private)

4.4.1.4. **MemberDao.java** : 회원(member) 데이터를 관리하는 DAO 클래스

- **create(Member member)** : member 테이블에 회원 정보를 INSERT하고 생성된 member_id를 반환한다
- **findById(Long id)** : member_id로 단일 회원을 조회
- **findByEmail(String email)** : 이메일로 회원 정보를 조회
- **map(ResultSet rs)** : ResultSet을 Member 객체로 변환한다 (private)

4.4.1.5. **MovieDao.java** : 영화(movie) 정보를 조회하는 DAO 클래스

- **findAll()** : 전체 영화 목록을 조회 (배급사 정보 JOIN, LIMIT 100)
- **findById(int movieId)** : movie_id 기준으로 단일 영화 정보를 조회

4.4.1.6. **MovieVersionDao.java** : 영화 버전(movie_version) 정보를 관리하는 DAO 클래스

- **findAll()** : 전체 영화 버전 목록을 조회
- **findByMovie(int movieId)** : 특정 영화의 버전 목록을 조회
- **findByFormat(String format)** : 특정 포맷(IMAX/4DX 등) 기준으로 버전 목록을 조회
- **mapRow(ResultSet rs)** : ResultSet을 MovieVersion 객체로 변환한다 (private)

4.4.1.7. **PaymentDao.java**

- 결제(payment) 데이터를 관리하는 DAO 클래스
- insert(Payment payment) : payment 테이블에 결제 정보를 INSERT하고 생성된 payment_id를 반환한다

4.4.1.8. PriceDao.java : 가격(price) 및 가격 계산 함수 정보를 관리하는 DAO 클래스

- findFinalPriceBySeat(Long showId, Long seatId) : MySQL 함수 fn_get_final_price를 호출하여 좌석 단위 최종 가격을 조회
- findFinalPrice(Long showId, Long seatTypeId) : price 테이블에서 seat_type 기준으로 상영 가격을 조회

4.4.1.9. ScreenDao.java : 상영관(screen) 데이터를 조회하는 DAO 클래스

- findByTheater(int theaterId) : 특정 영화관의 모든 상영관 목록을 조회
- findById(int screenId) : screen_id 기준으로 단일 상영관 정보를 조회

4.4.1.10. SeatDao.java : 좌석(seat) 및 좌석 상태를 조회하는 DAO 클래스

- findById(Long seatId) : seat_id 기준으로 단일 좌석을 조회
- findByLayout(Long layoutId) : 특정 layout에 속한 모든 좌석을 조회
- findSeatsByShow(Long showId) : 특정 상영에 대한 좌석 목록과 상태(BOOKED/HOLD/AVAILABLE)를 조회
- mapRowToSeat(ResultSet rs) : ResultSet을 Seat 객체로 변환한다 (private)
- getLayoutIdForShow(Long showId) : 상영 ID로 layout_id를 조회 (private)

4.4.1.11. ShowtimeDao.java : 상영시간(showtime) 정보를 조회하는 DAO 클래스

- findById(Long showId) : show_id 기준으로 단일 상영시간 정보를 조회
- findByMovieId(Long movieId) : 특정 영화의 상영시간 목록을 조회
- mapRowToShowtime(ResultSet rs) : ResultSet을 Showtime 객체로 변환 (private)

4.4.1.12. TheaterDao.java : 영화관(theater) 정보를 조회하는 DAO 클래스

- findAll() : 전체 영화관 목록을 조회
- findById(int theaterId) : theater_id로 단일 영화관을 조회

4.4.2. service

4.4.2.1. BookingService.java : 예약 생성 및 예약 프로세스 전체를 처리하는 비즈니스 로직 클래스

- BookingService() : BookingDao, BookingSeatDao, HoldSeatDao, PriceDao, SeatService를 초기화
- createBookingForMember(Long memberId, Long showId, List<Long> seatIds) : 회원의 예약을 생성. 좌석 가용성 확인, 가격 계산, booking/booking_seat INSERT, 홀드 삭제까지 포함

4.4.2.2. HoldSeatService.java : 좌석 임시 점유(홀드) 기능을 처리하는 비즈니스 로직 클래스

- HoldSeatService() : HoldSeatDao, BookingSeatDao를 초기화한다
- holdSeatForMember(Long memberId, Long showId, Long seatId) : 회원이 특정 좌석을 임시로 홀드. 이미 예약/홀드 된 좌석이면 실패
- cancelHold(Long holdId) : 특정 hold_id의 홀드를 취소
- clearExpiredHolds() : 만료된 홀드를 일괄 삭제

4.4.2.3. **MemberService.java** : 회원 등록 및 로그인 처리 등 회원 관련 비즈니스 로직 클래스

- **MemberService()** : MemberDao를 초기화
- **registerMember(String email, String name)** : 새로운 회원을 등록하고 member_id를 반환
- **login(String email)** : 이메일로 회원을 조회하여 반환
- **loginOrRegister(String email, String name)** : 로그인 또는 자동 회원가입 처리
- **getMemberById(Long id)** : member_id로 단일 회원 정보를 조회

4.4.2.4. **MovieService.java** : 영화 조회 관련 비즈니스 로직 클래스

- **getAllMovies()** : 전체 영화 목록을 조회
- **getMovieById(int movieId)** : 단일 영화 정보를 movie_id 기준으로 조회

4.4.2.5. **MovieVersionService.java** : 영화 버전(포맷) 조회 비즈니스 로직 클래스

- **getVersionsByMovie(int movieId)** : 특정 영화의 버전 목록을 조회
- **getVersionsByFormat(String format)** : IMAX/2D/4DX 등 특정 포맷의 버전 목록을 조회

4.4.2.6. **PriceService.java** : 가격 조회 비즈니스 로직 클래스

- **getFinalPrice(Long showId, Long seatTypeId)** : 상영 ID와 좌석 타입 ID로 최종 가격(BigDecimal)을 조회
- **getFinalPriceAsInt(Long showId, Long seatTypeId)** : 최종 가격을 int로 변환하여 반환

4.4.2.7. **ScreenService.java** : 상영관 조회 비즈니스 로직 클래스

- **getScreensByTheater(int theaterId)** : 특정 영화관의 상영관 목록을 조회
- **getScreen(int screenId)** : 단일 상영관 정보를 screen_id로 조회

4.4.2.8. **SeatService.java** : 좌석과 좌석 상태 조회를 처리하는 비즈니스 로직 클래스

- **getSeatsWithStatus(Long showId)** : 특정 상영의 좌석 목록과 상태(BOOKED/HOLD/AVAILABLE)를 조회 (-> :M seat, booking_seat, hold_seat 테이블을 동시에 조회하여 해당 상영 시간에 좌석 목록과 상태(BOOKED/HOLD/AVAILABLE)를 스냅샷처럼 반환.)
- **areSeatsAllAvailable(Long showId, List<Long> seatIds)** : 선택한 좌석들이 모두 예약 가능한지 검사

4.4.2.9. **ShowtimeService.java** : 상영시간 조회 비즈니스 로직 클래스

- **getShowtimesByMovie(Long movieId)** : 특정 영화의 상영시간 목록을 조회
- **getShowtime(Long showId)** : show_id 기준으로 단일 상영시간 정보를 조회

4.4.2.10. **TheaterService.java** : 영화관 조회 비즈니스 로직 클래스

- **getAllTheaters()** : 전체 영화관 목록을 조회
- **getTheater(int theaterId)** : theater_id 기준으로 단일 영화관 정보를 조회

4.4.3. model

Model 계층은 DB의 테이블과 매핑들로 구성되며, 예약(Booking), 좌석(Seat), 홀드(HoldSeat), 상영(Showtime), 영화(Movie), 회원(Member) 등 도메인 개념을 자바 객체로 표현하여 Service/DAO 계층이 공통된 자료구조를 사용할 수 있도록 한다. 대부분 필드, 생성자, getter/setter로 구성되며, 비즈니스 로직은 거의 포함하지 않는다

- **Booking.java** : 예약(booking) 테이블과 매핑
- **BookingSeat.java** : 예약 좌석(booking_seat) 테이블과 매핑
- **HoldSeat.java** : 좌석 홀드(hold_seat) 테이블과 매핑
- **Member.java** : 회원(member) 테이블과 매핑
- **Movie.java** : 영화(movie) 테이블과 매핑
- **MovieVersion.java** : 영화 버전(movie_version) 테이블과 매핑
- **Payment.java** : 결제(payment) 테이블과 매핑
- **Price.java** : 가격(price) 테이블과 매핑
- **Screen.java** : 상영관(screen) 테이블과 매핑
- **Seat.java** : 좌석(seat) 테이블과 매핑 **SeatStatus.java** : 좌석 상태를 나타내는 enum 클래스
- **Showtime.java** : 상영시간(showtime) 테이블과 매핑
- **Theater.java** : 영화관(theater) 테이블과 매핑

4.4.4. util

Util 계층은 애플리케이션 전체에서 공통으로 사용되는 기능을 제공한다. 데이터베이스 연결 및 Connection 생성을 전담하는 DBUtil 클래스로 구성되어 있다.

4.4.4.1. **DBUtil.java** : 데이터베이스 연결 유틸리티 클래스로 URL, USERNAME, PASSWORD - 환경변수(DB_URL, DB_USERNAME, DB_PASSWORD)에서 불러오는 DB 접속 정보로, getConnection() - DriverManager를 사용해 MySQL 데이터베이스와 연결

4.4.5. Main.java

Main 클래스는 콘솔 기반 영화 예매 시스템의 진입점으로서, 사용자 입력을 받아 서비스를 수행하고, 예매, 결제, 취소, 조회 등 전체 비즈니스 흐름을 제어한다.

scanner : 사용자 입력을 받기 위한 Scanner 객체

movieService, showtimeService, seatService, bookingService, memberService, theaterService, screenService : 도메인별 Service 객체들

paymentDao : 결제 처리를 위한 PaymentDao 객체

currentMember : 현재 로그인 중인 회원 정보를 저장하는 정적 필드

- **main(String[] args)** : 프로그램 진입점. mainMenuLoop()를 호출해 전체 메뉴
- **mainMenuLoop()** : 콘솔 기반 메인 메뉴를 반복 출력하며 기능 선택을 받음
- **describeCurrentUser()** : 현재 로그인한 회원 정보를 메뉴 상단에 출력
- **memberMenu()** : 회원 기능 메뉴로 이동. 이메일 로그인, 자동 회원가입 로그인, 신규 회원가입, 로그아웃을 처리
- **runMemberReservationFlow()** : 영화 예매 전체 흐름을 처리, 영화 선택 → 상영 선택 → 좌석 선택 → BookingService를 통한 예약 생성 순으로 진행
- **payForBooking()** : 예약 ID를 입력받아 결제를 진행하는 기능으로 결제가 성공하면 PaymentDao를 통해 payment 레코드를 생성하고 예약 상태를 CONFIRMED로 업데이트
- **viewMyBookings()** : 현재 로그인 회원의 모든 예매 내역을 조회·출력. BookingService 혹은 BookingDao를 활용해 회원 ID 기준으로 예약들을 로드함
- **cancelBookingWithRefund()** : 예매 취소 및 환불 처리 기능이 있으며, 상영 시작 20분 전까지는 100% 환불을 적용, 결제 취소 처리와 booking 상태 변경(CANCELLED)을 함께 수행
- **theaterScreenMenu()** : 영화관 목록을 출력하고 사용자가 선택한 영화관의 스크린(상영관) 목록을 조회하여 보여주는 기능
- **selectMovie()** : 영화 목록을 출력하고 사용자에게 영화 ID 입력을 받아 반환
- **selectShowtime(movieId)** : 특정 영화의 상영 시간 목록을 조회하여 출력하고 사용자가 선택한 showtime ID를 반환
- **selectSeats(showId)** : 해당 상영의 좌석 상태(BOOKED, HOLD, AVAILABLE)를 출력하고 사용자가 원하는 좌석 번호들 입력을 받아 리스트로 반환
- **printSeatLayout(List<Seat> seats)** : seats 리스트를 행(rowLabel), 열(colNumber) 기준으로 재구성하여 시각적으로 출력(예약 완료 좌석(X), 홀드 좌석(H), 예약 가능 좌석(공백))

4.5. 예외 처리 및 오류 대응

시스템의 예외 처리는 입력 오류, 비즈니스 로직 오류, 데이터베이스 예외의 세 범주로 구분하여 최소한의 흐름 제어가 가능하도록 구성하였다.

4.5.1. 입력 오류

- 메뉴 선택 시 유효하지 않은 번호가 입력되면 메시지를 출력하고 동일 메뉴를 다시 표시한다.
- 영화 ID·상영 ID 입력 시 숫자 여부를 검사하고, selectMovie()와 selectShowtime()에서는 미리 준비한 유효 ID 목록에 포함되지 않으면 재입력을 요구한다.
- 좌석 선택 단계에서도 빈 입력 또는 잘못된 형태의 입력이 들어올 경우 예매 흐름이 종료되게 하였다.

4.5.2. 비즈니스 로직 오류

- 예매 규칙 위반은 Service 계층에서 처리한다.
- 로그인되지 않은 상태에서는 예매·결제·취소 기능을 수행할 수 없도록 하였다(구현상 제약점이다.)
- 좌석 선택 시 SeatService.areSeatsAllAvailable()로 사용 여부를 확인하고, 이미 예약되었거나 다른 사용자가 홀드 중인 좌석은 BookingService 내부 검사를 통해 즉시 차단한다.
- 가격 정보가 존재하지 않는 경우 PriceDao가 null을 반환하므로 해당 좌석에 대한 예약을 중단한다.
- 결제·취소 기능은 예약 상태를 확인하여 예약에 대해 중복 처리가 발생하지 않도록 조건 검사를 수행한다.

4.5.3. 데이터베이스 예외

- DB 연결은 DBUtil에서 생성하며 실패 시 SQLException을 전달한다.
- DAO 계층은 대부분 예외를 그대로 throws하고, 일부 조회 기능은 내부에서 처리 후 null 또는 빈 컬렉션을 반환한다.
- Service 계층에서는 DAO 호출을 try-catch로 오류 메시지를 출력하고 흐름을 종료하는 방식으로 일관되게 처리한다.
- 결제 및 환불 처리에서는 필요한 구간에 트랜잭션을 적용하여, 예외 발생 시 rollback()한다.

5. 테스트 및 결과(Test / Result)

5.1. 메뉴

```
===== 영화 예매 시스템 (콘솔 버전) =====

현재 로그인 상태: 로그인되지 않음
=====

1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요:
```

5.2. 회원가입

```
===== 회원 메뉴 =====
1. 이메일로 로그인
2. 이메일 + 이름으로 로그인 (없으면 자동 회원가입)
3. 새 회원 가입
4. 로그아웃
0. 이전 메뉴로
선택: 3
이메일을 입력하세요: login@cau.ac.kr
이름을 입력하세요: login
회원 가입 완료! 회원 ID=254
```

5.3. 중복 회원가입

```
-----
현재 로그인 상태: 로그인되지 않음
-----

1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 1

===== 회원 메뉴 =====
1. 이메일로 로그인
2. 이메일 + 이름으로 로그인 (없으면 자동 회원가입)
3. 새 회원 가입
4. 로그아웃
0. 이전 메뉴로
선택: 3
이메일을 입력하세요: test@cau.ac.kr
이름을 입력하세요: test
[ERROR] 회원 등록 실패: Duplicate entry 'test@cau.ac.kr' for key 'member.email'
회원 가입에 실패했습니다.
```

5.4. 로그인

```
===== 회원 메뉴 =====
1. 이메일로 로그인
2. 이메일 + 이름으로 로그인 (없으면 자동 회원가입)
3. 새 회원 가입
4. 로그아웃
0. 이전 메뉴로
선택: 1
이메일을 입력하세요: test@cau.ac.kr
로그인 성공! 회원 ID=251, 이름=test
```

5.5. 자동 회원가입

```
===== 회원 메뉴 =====
1. 이메일로 로그인
2. 이메일 + 이름으로 로그인 (없으면 자동 회원가입)
3. 새 회원 가입
4. 로그아웃
0. 이전 메뉴로
선택: 3
이메일을 입력하세요: user@cau.ac.kr
이름을 입력하세요: user
회원 가입 완료! 회원 ID=253
```

5.6. 로그아웃

```
===== 회원 메뉴 =====
1. 이메일로 로그인
2. 이메일 + 이름으로 로그인 (없으면 자동 회원가입)
3. 새 회원 가입
4. 로그아웃
0. 이전 메뉴로
선택: 4
로그아웃되었습니다.
```

5.7. 영화 예매 - 상영중

```

-----
현재 로그인 상태: [회원] ID=251, 이메일=test@cau.ac.kr
-----
1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 2

===== 영화 목록 =====
1. Movie 1 (상영시간: 142분, 관람등급: 1)
2. Movie 2 (상영시간: 116분, 관람등급: 4)
3. Movie 3 (상영시간: 123분, 관람등급: 1)
4. Movie 4 (상영시간: 141분, 관람등급: 1)
5. Movie 5 (상영시간: 142분, 관람등급: 5)
6. Movie 6 (상영시간: 99분, 관람등급: 5)
7. Movie 7 (상영시간: 90분, 관람등급: 1)
8. Movie 8 (상영시간: 133분, 관람등급: 5)
9. Movie 9 (상영시간: 108분, 관람등급: 5)
10. Movie 10 (상영시간: 137분, 관람등급: 3)
11. Movie 11 (상영시간: 134분, 관람등급: 5)
12. Movie 12 (상영시간: 87분, 관람등급: 1)
13. Movie 13 (상영시간: 107분, 관람등급: 1)
14. Movie 14 (상영시간: 110분, 관람등급: 5)
15. Movie 15 (상영시간: 100분, 관람등급: 4)

```

5.8. 영화 예매 - 상영 정보, 좌석, 예약 완료

```

===== 상영 정보 목록 =====
141. 상영관ID=24, 시작=2023-02-07 16:10, 종료=2023-02-07 18:10
159. 상영관ID=8, 시작=2023-02-12 17:14, 종료=2023-02-12 19:14
예매할 상영 ID를 선택하세요 (0 입력 시 취소): 141
선택한 상영 ID: 141

===== 좌석 현황 =====

A행: 461[ ] 462[ ] 463[ ] 464[ ] 465[ ]
B행: 466[ ] 467[ ] 468[ ] 469[ ] 470[ ]
C행: 471[ ] 472[ ] 473[ ] 474[ ] 475[ ]
D행: 476[ ] 477[ ] 478[ ] 479[ ] 480[ ]

[X]=예약됨, [H]=홀드, [ ]=예약 가능
예약 가능한 좌석의 seat_id를 콤마(,)로 구분해서 입력하세요.
예: 1,2,3 (0 입력 시 취소)
461, 462
선택한 좌석 ID 목록: [461, 462]
[INFO] 예약 생성 완료. bookingId=252, totalAmount=34507.00

===== 예약 완료 =====
예약 ID      : 252
회원 ID      : 251
상영 ID      : 141
총 결제 금액 : 34507.00
상태         : PENDING
생성 시각    : null

```

5.9. 결제

```
* 결제 메뉴에서 결제를 진행할 수 있습니다.

-----
현재 로그인 상태: [회원] ID=251, 이메일=test@cau.ac.kr
-----

1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 3
결제할 예약 ID를 입력하세요: 252
예약 ID=252, 결제 금액=34507.00, 현재 상태=PENDING
결제 수단 ID를 입력하세요 (예: 1=카드, 2=계좌이체 등):
1
결제 완료! paymentId=231
예약 상태가 CONFIRMED로 변경되었습니다.
```

5.10. 예약 좌석

```
===== 상영 정보 목록 =====
141. 상영관ID=24, 시작=2023-02-07 16:10, 종료=2023-02-07 18:10
159. 상영관ID=8, 시작=2023-02-12 17:14, 종료=2023-02-12 19:14
예매할 상영 ID를 선택하세요 (0 입력 시 취소): 141
선택한 상영 ID: 141

===== 좌석 현황 =====

A행: 461[X] 462[X] 463[ ] 464[ ] 465[ ]
B행: 466[ ] 467[ ] 468[ ] 469[ ] 470[ ]
C행: 471[ ] 472[ ] 473[ ] 474[ ] 475[ ]
D행: 476[ ] 477[ ] 478[ ] 479[ ] 480[ ]

[X]=예약됨, [H]=홀드, [ ]=예약 가능
예약 가능한 좌석의 seat_id를 콤마(,)로 구분해서 입력하세요.
예: 1,2,3 (0 입력 시 취소)
```

5.11. 예매 내역

```
-----
현재 로그인 상태: [회원] ID=251, 이메일=test@cau.ac.kr
-----

1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 4

===== 내 예매 내역 =====
예약ID=252, 상영ID=141, 상태=CONFIRMED, 금액=34507.00, 생성시각=2025-11-30T19:48:37
예약ID=251, 상영ID=9, 상태=PENDING, 금액=29361.00, 생성시각=2025-11-29T20:12:28
```

5.12. 예매 취소

```
-----
현재 로그인 상태: [회원] ID=251, 이메일=test@cau.ac.kr
-----
1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 5
취소할 예약 ID를 입력하세요: 252

===== 예매 취소 처리 완료 =====
예약 ID      : 252
환불 금액    : 0
취소 상태    : CANCELLED
```

5.13. 상영관 및 스크린 조회

```
-----
현재 로그인 상태: [회원] ID=251, 이메일=test@cau.ac.kr
-----
1. 회원 로그인 / 회원가입
2. 영화 예매
3. 결제
4. 내 예매 내역 조회
5. 예매 취소 및 환불
6. 상영관 및 스크린 조회
0. 종료
메뉴를 선택하세요: 6

===== 상영관 목록 =====
1. Theater 1 (Address 1)
2. Theater 2 (Address 2)
3. Theater 3 (Address 3)
4. Theater 4 (Address 4)
5. Theater 5 (Address 5)
6. Theater 6 (Address 6)
7. Theater 7 (Address 7)
8. Theater 8 (Address 8)
9. Theater 9 (Address 9)
10. Theater 10 (Address 10)
11. Theater 11 (Address 11)
12. Theater 12 (Address 12)
13. Theater 13 (Address 13)
14. Theater 14 (Address 14)
15. Theater 15 (Address 15)
상세 조회할 상영관 ID를 입력하세요 (0 입력 시 취소): 1

선택한 상영관: Theater 1 (Address 1)

===== 스크린 목록 =====
- 스크린 ID=1, 이름=Screen 1
- 스크린 ID=2, 이름=Screen 2
```

6. 결론 및 개선점(Conclusion)

본 프로젝트는 영화 예매 도메인을 바탕으로, 데이터베이스 설계 및 DB 프로그래밍, 그리고 비즈니스 로직 구현을 목표로 개발되었다. 전체 시스템은 콘솔 환경에서 동작하도록 설계되었으며, 회원 관리, 영화 및 상영 정보 조회, 예매, 결제, 환불 등 실제 영화관의 핵심 업무 흐름을 묘사하였다.

특히 좌석 점유 충돌을 방지하기 위해 'hold'메커니즘을 도입하였으며, 예약 상태 검증 로직과 더불어 함수, 프로시저, 트리거를 활용함으로써 기능 구현뿐만 아니라 데이터 정합성을 확보하는 데 기여하였다.

프로젝트 진행 과정에서 시간 및 인력 부족으로 인해 발생한 제약점과, 시스템적인 한계점은 다음과 같다.

- **환경적 제약** : 콘솔 기반으로 구현되어 실제 사용자가 겪는 UX(User Experience)와는 괴리가 있으며, 로컬 DB 환경에 의존하여 확장성과 배포 측면에서 제약이 존재한다.
- **구현 범위의 한계** : 전체 ERD(42개 테이블)는 실제 서비스를 기준으로 설계되었으나, 개발 리소스의 한계로 인해 추천 시스템, 스낵 관리 등 일부 기능은 실제 구현 단계에서 제외되었다.
- **기술적 한계** : 트랜잭션 관리가 일부 핵심 기능에만 적용되어 전체 프로세스의 원자성(Atomicity)을 완

벽히 보장하지 못하며, **동시성 제어 또한 기본적인 수준에 머물러 있어** 대규모 트래픽 환경에는 적합하지 않다. 표준화된 예외 처리와 로깅 시스템의 부재 또한 해결해야 할 과제이다.

본 프로젝트는 학습 목적으로 수행되었으나, 설계 자체는 롯데시네마, CGV, 메가박스 등 실제 상용 플랫폼을 참고하여 진행되었기에 구조적인 확장 가능성을 가지고 있다.

향후 프로젝트를 발전시킨다면 **웹 기반 UI를 적용하고 클라우드 데이터베이스로 전환하여** 접근성을 높일 수 있을 것이다. 또한 이번에 구현하지 못했던 **추천 시스템이나 매점 및 영화관 운영 관리기능을 추가하고, 트랜잭션 및 동시성 제어 로직을 고도화**한다면 실제 서비스 수준의 시스템으로 발전시킬 수 있을 것이다.